
Connected Maximum-Density Still Lives in Conway’s Game of Life via a Verifier-Grounded Agentic Loop

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study the connected maximum-density still life problem in Conway’s Game
2 of Life—maximizing the number of live cells in a stable, 8-connected pattern
3 inside an $n \times n$ box—a problem from the CAISc 2026 verifiable-problems list.
4 The unconstrained variant is completely solved, but to the best of our knowledge
5 no per- n table of *connected* optima has previously been published. We attack
6 the problem with a verifier-grounded agentic loop: a large language model or-
7 chestrates a grader-faithful deterministic verifier, two independent exact engines
8 (a CP-SAT model cross-checked under two independent connectivity encodings,
9 single-commodity flow and spanning tree, and an independently reproduced answer-
10 set-programming encoding), and a stochastic proposer, re-routing among them
11 based on observed solver behavior. The verifier validates every claimed pattern,
12 so no reported result rests on solver trust alone. We contribute the first per- n con-
13 nected table for the small boxes: connected optima *proven* for $n \in \{8, 9, 10, 11\}$
14 (respectively 32, 43, 53, and 63 live cells), and verifier-validated best-found values
15 with two-sided bounds for $n \in \{13, 15\}$, and verifier-validated best-found patterns
16 for $16 \leq n \leq 21$ from a budgeted constraint-solver heuristic. We also report three
17 methodological findings: a sandwich “ceiling-matching” argument that proves op-
18 timality without exhaustive search whenever a connected witness meets the proven
19 unconstrained optimum; evidence that the CP-SAT flow encoding outperforms
20 the ASP reachability encoding on proof closure for $n \geq 10$; and a diagnosis of
21 why naive verify-or-reject simulated annealing fails on this problem. All witness
22 patterns and the verifier are released as machine-checkable artifacts.

23 1 Introduction

24 Conway’s Game of Life is a two-state cellular automaton on the square grid in which a live cell
25 survives with two or three live neighbors and a dead cell is born with exactly three. A *still life* is a
26 pattern fixed under this rule. The maximum-density still life problem—how many live cells fit in an
27 $n \times n$ box without instability—is a classic constraint-optimization benchmark and was completely
28 solved by Chu and Stuckey [1], whose values coincide with OEIS sequence A055397 [2]. Those
29 optima are *pseudo* still lifes: they need not be connected, and for several box sizes the densest stable
30 pattern is in fact a regular array of disjoint blocks.

31 The *connected* variant adds a single global constraint—all live cells must form one island under
32 8-adjacency—and is the subject of this paper and of the corresponding CAISc 2026 verifiable
33 problem [12]. Connectivity changes the optimization in a way that is poorly documented: it appears
34 only as a benchmark in the ASP Competition [8], which measured solver performance rather than
35 publishing per- n optima, and to the best of our knowledge—after searching the standard still-life

36 references, community censuses, and competition reports—no maintained per- n table of connected
37 optima or best-known values exists. This gap is the opportunity.

38 CAISc rewards the AI-driven methodology as much as the combinatorial result. Our primary
39 contribution is therefore an *agentic discovery loop* and its application, not merely a list of patterns:

- 40 • **A verifier-grounded agentic architecture.** A large language model orchestrates a grader-
41 faithful deterministic verifier, two independent exact engines (CP-SAT with a single-
42 commodity-flow connectivity encoding; a reproduced ASP reachability encoding), and
43 a stochastic proposer. The deterministic verifier is foundational: every reported pattern is
44 re-checked against it, so validity never depends on trusting a solver or the agent.
- 45 • **The first per- n connected table for small boxes.** We prove the connected optimum for
46 $n \in \{8, 9, 10, 11\}$ and give verifier-validated best-found values with two-sided bounds for
47 $n \in \{13, 15\}$; for $16 \leq n \leq 21$ we report verifier-validated best-found patterns.
- 48 • **Methodological findings.** (i) A “ceiling-matching” sandwich argument (Proposition 1) that
49 proves optimality without exhaustive search whenever a connected witness meets the proven
50 unconstrained ceiling. (ii) Evidence that, on the connectivity-augmented Life encoding, a
51 CP-SAT flow formulation closes proofs that an ASP reachability formulation cannot for
52 $n \geq 10$. (iii) A diagnosis of why naive verify-or-reject simulated annealing makes no valid
53 progress on this problem, which is itself an explanation for why the literature reaches for
54 exact and hybrid methods.

55 **Claims and scope.** We claim proven optimality only for $n \in \{8, 9, 10, 11\}$; all other reported
56 values are explicitly labeled best-found, with bounds where available. Every claimed pattern is
57 validated by the released verifier in the official submission format. We make no claim of having
58 found the connected optimum for $n \geq 13$.

59 2 Problem definition and background

60 **Definition.** Fix a box size n . A configuration is an assignment of $\{0, 1\}$ to the cells of an $n \times n$
61 grid. Writing $N(i, j)$ for the number of live cells among the eight Moore neighbors of (i, j) , the
62 configuration is a *connected still life* iff: (1) every live cell (i, j) has $N(i, j) \in \{2, 3\}$ (stability); (2)
63 no dead cell has $N(i, j) = 3$, where this birth condition is checked for every cell of the box *and*
64 of its one-cell exterior ring (no births); and (3) the live cells form a single 8-connected component.
65 The objective is to maximize the number of live cells. We denote the connected optimum by
66 $C\text{-OPT}(n)$ and the unconstrained optimum by $a(n) = A055397(n)$. Permitted box sizes are $n \in$
67 $\{8, 9, 10, 11, 13, 15, 16, 17, 19, 20, 21, 31, 32\}$ [12].

68 **The ceiling.** Because every connected still life is a still life, restricting to connected patterns can
69 only shrink the feasible set, so $C\text{-OPT}(n) \leq a(n)$ for all n . The values $a(n)$ are proven optimal [1]
70 and serve as our upper bounds throughout (Table 1). They are attained by possibly-disconnected
71 patterns; for box sizes of the form $3k - 1$ (including $n = 8$ and $n = 11$ among our set) the densest
72 stable configuration is a disjoint block array, so a strict gap $C\text{-OPT}(n) < a(n)$ is expected there.
73 Elkies [3] proved that the infinite-lattice still-life density is at most $1/2$; finite boxes can exceed this,
74 and our connected densities sit just above 50%, consistent with that bound.

75 **Related work.** Exact approaches to the unconstrained problem include hybrid integer/constraint
76 programming [4], variable (bucket) elimination [5], dedicated constraint models [6], and the lazy-
77 clause-generation solution that closed the problem [1]; memetic/exact hybrids gave strong large-
78 instance results [7]. Enforcing connectivity in combinatorial models is itself well studied—single-
79 and multi-commodity flow, spanning-tree, and lazy separator-cut formulations are standard tools.
80 Methodologically, our loop follows recent work pairing generative models with deterministic eval-
81 uators for discovery: program search with an evaluator [9], evolutionary coding agents [10], and
82 solver-evolution under strict verification [11], the last of which emphasizes that a precisely calibrated
83 verifier is foundational rather than optional—a principle we adopt directly.

84 3 The agentic discovery loop

85 3.1 Architecture and the foundational verifier

86 The system has four components. A **deterministic verifier** implements the three conditions of
87 Section 2 exactly, including the one-cell exterior-ring birth check and an 8-connectivity breadth-first
88 search, in $O(n^2)$. Two **exact engines** (Sections 3.2–3.3) search for and prove optima. A **stochastic**
89 **proposer** (Section 3.5) targets the larger boxes. A **language-model orchestrator** writes and revises
90 the encodings and operators, routes candidates, and decides when to switch strategies. Crucially, the
91 verifier sits between every engine and every reported result: a pattern counts only if it passes the
92 verifier, and the verifier is validated against tiny known still lifes (block, beehive) and negative cases
93 (a disconnected pair, an oscillating blinker) before use. No claim in this paper depends on trusting a
94 solver or the orchestrator.

95 3.2 Exact engine: CP-SAT with two independent connectivity encodings

96 We model each cell with a Boolean live variable. Stability and the (interior and exterior-ring) no-birth
97 conditions are posted as linear constraints on neighbor sums that mirror the verifier. Our primary
98 connectivity encoding is a single-commodity flow on the 8-adjacency graph: a virtual super-source
99 injects a number of units equal to the total live count into one model-chosen *root*; the root indicator
100 is free with $\sum \text{root} \leq 1$ and is forced to coincide with a live cell; each live cell consumes exactly
101 one unit; and arc flow is gated so that flow may traverse an arc only if both endpoints are live. A
102 configuration then routes feasibly iff every live cell is reachable from the root, i.e. iff it is connected.
103 The model adds $O(n^2)$ flow variables and is solved with the objective of maximizing live cells.

104 Because a subtle error in the connectivity encoding is the most plausible way an apparent “proof”
105 could be wrong, we implemented a second, structurally different connectivity model in the same
106 solver and required the two to agree. In the spanning-tree encoding each live cell selects a parent
107 among its live neighbors, exactly one live cell is the parentless root, and an integer depth variable
108 strictly increases from parent to child; this admits a rooted tree—and so certifies connectivity—iff the
109 live set is a single component, with no flow conservation involved. The two encodings share identical
110 stability and no-birth constraints and differ only in the connectivity mechanism (global flow vs. acyclic
111 parent pointers). On $n \in \{8, 10, 11\}$ they return the *same* optimum from *different* witness patterns,
112 and the spanning-tree model was consistently the faster of the two (for example 1.3 vs. 43.6 s at $n = 8$,
113 5.0 vs. 44.0 s at $n = 10$, and 116 vs. 323 s at $n = 11$). Agreement of two independent encodings,
114 each emitting its own optimality certificate, effectively eliminates the connectivity-modeling failure
115 mode. Together this engine closed $n \in \{8, 9, 10, 11\}$ to proven optimality within minutes.

116 3.3 Independent cross-checks: a reproduced ASP encoding and ceiling enumeration

117 We additionally cross-checked the exact results with answer-set programming, a different solver
118 family. First, we reproduced the connected-still-life ASP encoding of the competition benchmark [8],
119 which expresses connectivity through recursive reachability from the lexicographically-minimal live
120 cell; reproducing it surfaced a silent semantic bug (Section 3.6), and once corrected the ASP and
121 CP-SAT engines agree on every value they both decide, with the ASP solver independently exhausting
122 the search at $n = 8$ (and $n = 9$ settled by ceiling-matching). Second, to probe the $n = 10$ and
123 $n = 11$ upper bounds without any connectivity encoding at all, we ran a *stability-only* enumeration:
124 we removed every connectivity rule, fixed the live count to the ceiling $a(n)$, enumerated the resulting
125 still lifes, and tested each for 8-connectivity with the verifier. At $n = 8$ this terminated and showed
126 the unique 36-cell still life to be disconnected, independently confirming $C\text{-OPT}(8) < 36$. At $n = 10$
127 and $n = 11$ it enumerated 364 and 2 ceiling-density still lifes respectively—all disconnected—but
128 did not exhaust the space within budget, so it strongly corroborates the CP-SAT optimality result
129 for those two boxes without, by itself, constituting an independent proof. Across solver family and
130 connectivity mechanism the formulations agree wherever they both decide, which we read as genuine
131 triangulation rather than a single tool checked against itself.

132 3.4 Ceiling-matching: optimality without exhaustion

133 The orchestrator adopted a simple but useful proof rule that avoids redundant search.

134 **Proposition 1** (Ceiling-matching). *For every n , $C\text{-OPT}(n) \leq a(n)$. If a verifier-validated connected*
 135 *still life with $a(n)$ live cells exists, then $C\text{-OPT}(n) = a(n)$.*

136 *Proof.* Every connected still life is a still life, so the connected feasible set is a subset of the
 137 unconstrained one and $C\text{-OPT}(n) \leq a(n)$. A verified connected witness with $a(n)$ cells gives
 138 $C\text{-OPT}(n) \geq a(n)$. The two inequalities give equality. $\square$

139 The argument is elementary; its value is operational. Whenever a connected witness reaches the
 140 published ceiling, optimality follows immediately from the proven unconstrained result [1] and no
 141 exhaustive proof is needed—this is how $n = 9$ was closed in under two seconds. The rule is sound
 142 only at equality: a witness strictly below the ceiling proves nothing about optimality, and those cases
 143 require true exhaustive search (Section 4).

144 3.5 The stochastic proposer, and why naive simulated annealing fails

145 For boxes beyond the reach of exact proof we implemented a simulated-annealing proposer with
 146 verify-or-reject acceptance, seeded from dense disconnected agars and from tilings of small proven
 147 patterns. On this problem the naive version makes essentially no progress: from any near-optimal
 148 seed, single-cell toggles and small-window edits almost never yield a valid configuration, because
 149 Life’s stability constraints are extremely tight—an added isolated cell is unstable, a removed cell
 150 destabilizes its neighbors, and most local changes induce a birth in or beside the edited region or
 151 sever connectivity. We document this failure rather than hide it: it is a concrete explanation for why
 152 the literature on dense still lifes relies on exact search and on memetic/exact hybrids with bucket-
 153 elimination repair [7] rather than vanilla local search, and it motivates structured neighborhoods (e.g.
 154 inserting stable 2×2 atoms, sliding blocks) or large-neighborhood repair with an exact sub-solver as
 155 the correct heuristic design.

156 3.6 Agent trajectory and orchestration

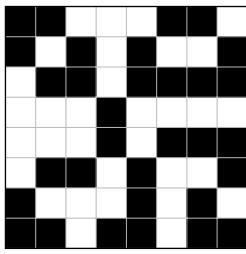
157 The development trajectory is part of the contribution. Three episodes are illustrative. (i) The
 158 reproduced ASP encoding contained an aggregate whose neighbor-counting term collapsed under
 159 modern stable-model semantics, so the only satisfying configuration was the empty grid; the bug was
 160 silent and was caught *only* because the verifier rejected the (nonexistent) progress, after which the
 161 orchestrator restored the intended counting semantics. (ii) An initial misreading of the exact solver’s
 162 optimality flag produced an apparent “best-found, not proven” result whose wall-clock was far below
 163 the time limit; the contradiction prompted a corrected reading of the solver’s search-exhausted signal.
 164 (iii) Confronted with the simulated-annealing stall (Section 3.5), the orchestrator re-routed effort to
 165 the exact engines for the small boxes and diagnosed the cause rather than blindly retuning the cooling
 166 schedule. Each episode is a case of the deterministic verifier or an internal consistency check catching
 167 an error that the generative component introduced—evidence for the verifier-grounded design.

168 4 Results and analysis

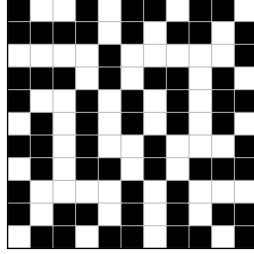
169 Table 1 reports the per- n connected results. For $n \in \{8, 9, 10, 11\}$ the connected optimum is proven;
 170 densities sit just above 50%. The connectivity penalty $a(n) - C\text{-OPT}(n)$ is 4 at $n = 8$ (where
 171 the unconstrained optimum is nine disjoint blocks), 0 at $n = 9$, and 1 at $n = 10$ and $n = 11$.
 172 For $n \in \{13, 15\}$ we give the best verified pattern together with a two-sided bound; for $n = 15$
 173 the solver’s own upper bound was slack, so we use the tighter unconstrained ceiling of Chu and
 174 Stuckey [1] as the upper limit. For $16 \leq n \leq 21$, exact proof is out of reach on our hardware, so we
 175 report verifier-validated best-found patterns from CP-SAT run as a time-budgeted heuristic; seeding
 176 the search with a dense block-lattice hint—which supplies the correct high-density macrostructure
 177 and lets the solver spend its budget on the connectivity-induced adjustments at block boundaries—
 178 improved these values by 15–105 cells over an unstructured embedding. In these runs the two boxes
 179 that improved over their earlier best ($n = 17$ and $n = 20$) did so under the block-lattice hint, while
 180 warm-starting from the previous best tended to re-discover it—a small signal that hint diversity, not
 181 only hint quality, matters for constraint-solver heuristics on these tight constraints. Across $n \in [8, 21]$
 182 the connected density stays close to one-half: the proven optima for $n \leq 11$ realize 50–53%, and
 183 the best-found patterns for $13 \leq n \leq 21$ realize roughly 50–52% (lower bounds on the true optima),

Table 1: Per- n connected maximum-density still life results. Ceilings $a(n)$ are the proven unconstrained optima [1, 2] and are upper bounds for $C\text{-OPT}(n)$. “Proven” values are connected optima; “best-found” values are verifier-validated lower bounds with the stated interval. All patterns pass the released verifier in the official submission format. Values for $16 \leq n \leq 21$ are verifier-validated best-found lower bounds from a budgeted heuristic and may improve with longer search.

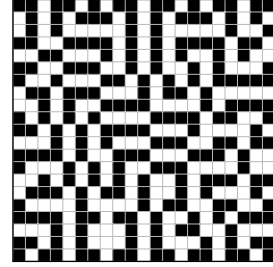
n	$a(n)$	connected	status	density	proof path
8	36	32	proven optimal	50.00%	CP-SAT search-exhausted; ASP exhausted
9	43	43	proven optimal	53.09%	ceiling-matching (Prop. 1)
10	54	53	proven optimal ¹	53.00%	CP-SAT optimal, two encodings
11	64	63	proven optimal ¹	52.07%	CP-SAT optimal, two encodings
13	90	88	best-found $\in \{88, 89\}$	52.07%	CP-SAT incumbent 88 (3 strategies), bound 89
15	119	116	best-found $\in [116, 119]$	51.56%	CP-SAT incumbent 116; ceiling 119
16	136	131	best-found	51.17%	CP-SAT + block-lattice hint
17	152	147	best-found	50.87%	CP-SAT + block-lattice hint
19	190	179	best-found	49.58%	CP-SAT + block-lattice hint
20	210	201	best-found	50.25%	CP-SAT + block-lattice hint
21	232	221	best-found	50.11%	CP-SAT + block-lattice hint



(a) $n = 8$, 32 cells (proven)



(b) $n = 11$, 63 cells (proven)



(c) $n = 21$, 221 cells (best-found)

Figure 1: Representative verifier-validated connected still lifes. (a,b) proven connected optima at $n = 8$ and $n = 11$; (c) a best-found pattern at $n = 21$. Filled squares are live cells; in every panel the pattern is stable, induces no birth in the box or its one-cell exterior ring, and forms a single 8-connected component. The $n = 8$ and $n = 11$ panels are the witnesses underlying the proven entries of Table 1; all three pass the released verifier in the official submission format.

consistent with the asymptotic $1/2$ density bound of Elkies [3] and indicating that connectivity does not push achievable density far below the unconstrained regime even at modest finite n .

Solver comparison. On this problem the two exact engines behaved differently as n grew. The ASP reachability encoding found tight witnesses quickly for $n \leq 9$ but stalled on both improvement and proof for $n \geq 10$ (for $n = 11$ its best was 62 versus the true optimum 63). The CP-SAT flow encoding closed $n = 8, 9, 10, 11$ to proven optimality, $n = 11$ in roughly five minutes and the others in under one minute on four performance cores. We read this as a datapoint about solver families on the connectivity-augmented Life encoding rather than a universal claim: lazy-clause-generation search with the flow connectivity model handled the global constraint more effectively than recursive reachability here. Within CP-SAT, the two connectivity encodings reached identical optima but differed sharply in cost—the spanning-tree formulation was 3–30 $\times$ faster than single-commodity flow on $n \in \{8, 10, 11\}$ —which we attribute to its more local propagation (parent pointers and a depth potential) relative to global flow conservation.

¹Optimality at $n = 10, 11$ is established by CP-SAT exhaustive-search certificates under *two* structurally independent connectivity encodings—single-commodity flow and a rooted spanning tree—which return the same optimum from different witnesses, eliminating a connectivity-encoding error as a failure mode. A separate ASP stability-only enumeration at the ceiling found only disconnected configurations (364 at $n = 10$, 2 at $n = 11$) but did not exhaust the space within budget, so it corroborates rather than independently proves these two upper bounds.

5 Limitations

Our proven results stop at $n = 11$; for $n = 13$ and $n = 15$ we report bounded best-found values, and for $16 \leq n \leq 21$ best-found patterns whose gap to the ceiling we do not claim to have closed. Exact proof at larger n was limited by the hardness of connectivity-constrained search on a single passively-cooled laptop within a fixed time budget, not by any closed-form barrier. Our naive simulated-annealing proposer did not produce valid moves and we did not have time to implement the structured-neighborhood or large-neighborhood-repair variants that Section 3.5 argues are needed; consequently the large- n entries are weaker than the small- n ones. The optimality of $n = 10$ and $n = 11$ is supported by two structurally independent connectivity encodings within CP-SAT (flow and spanning tree) that agree on the optimum from different witnesses, which eliminates a connectivity-modeling error; a fully solver-independent confirmation remains partial, since our ASP stability-only enumeration at the ceiling found only disconnected configurations but did not exhaust the space for these two boxes within budget. We judge the residual risk low, but a terminating non-CP-SAT optimality proof for $n = 10, 11$ would close the gap entirely. We did not attempt $n = 31$ or $n = 32$ within budget and exclude them from our reported results. Finally, all upper bounds inherit the correctness of the published unconstrained optima [1].

6 Conclusion

We presented a verifier-grounded agentic loop and used it to produce, to our knowledge, the first per- n table of connected maximum-density still lifes for the small boxes, with proven optima for $n \in \{8, 9, 10, 11\}$ and bounded best-found values beyond. The methodological yield—the ceiling-matching proof rule, the comparison of flow, spanning-tree, and reachability connectivity encodings, and the diagnosis of why naive local search fails—is, for an AI-for-science venue, as much the point as the patterns. We release the verifier, both CP-SAT connectivity encodings, the corrected ASP encoding, and all witness artifacts so that every reported value is independently machine-checkable.

References

References

- [1] G. Chu and P. J. Stuckey. A complete solution to the maximum density still life problem. *Artificial Intelligence*, 184–185:1–16, 2012.
- [2] OEIS Foundation Inc. Sequence A055397: Maximum population of an $n \times n$ stable pattern in Conway’s Game of Life. *The On-Line Encyclopedia of Integer Sequences*.
- [3] N. D. Elkies. The still-life density problem and its generalizations. arXiv:math/9905194, 1998.
- [4] R. Bosch and M. Trick. Constraint programming and hybrid formulations for three Life designs. *Annals of Operations Research*, 130:41–56, 2004.
- [5] J. Larrosa, E. Morancho, and D. Niso. On the practical use of variable elimination in constraint optimization problems: ‘still-life’ as a case study. *Journal of Artificial Intelligence Research*, 23:421–440, 2005.
- [6] G. Chu, K. E. Petrie, and N. Yorke-Smith. Constraint programming to solve maximal density still life. In *Game of Life Cellular Automata*, chapter 10, pages 99–114. Springer, 2010.
- [7] J. E. Gallardo, C. Cotta, and A. J. Fernández. Finding still lifes with memetic/exact hybrid algorithms. arXiv:0812.4170, 2008.
- [8] M. Alviano et al. The Fourth Answer Set Programming Competition: Preliminary report. In *Logic Programming and Nonmonotonic Reasoning (LPNMR 2013)*, LNCS 8148, pages 42–53. Springer, 2013. Connected Maximum-Density Still Life benchmark; Official Problem Suite, <https://www.mat.unical.it/aspcmp2013/>.
- [9] B. Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [10] A. Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025.

- 244 [11] C. Yu, R. Liang, C.-T. Ho, and H. Ren. Autonomous code evolution meets NP-completeness.
245 arXiv:2509.07367, 2025.
- 246 [12] Curated by Siddhartha Mahajan (2026). Connected Still Life. Reviewed by Dr. Ankan Pal. CAISc 2026
247 Verifiable Problems Track. [https://caisc2026.github.io/verifiable-problems/?problem=](https://caisc2026.github.io/verifiable-problems/?problem=connected-still-life)
248 [connected-still-life](https://caisc2026.github.io/verifiable-problems/?problem=connected-still-life).

Conference For AI Scientists 2026 - AI Involvement Checklist

Research Stage Assessment

For items 1-4, give a score from the scale below that defines the role of AI in each part of the scientific process. The scores are as follows:

- **[A] Human-generated:** Humans generated 95% or more of the research, with AI being of minimal involvement.
- **[B] Mostly human, assisted by AI:** The research was a collaboration between humans and AI models, but humans produced the majority (>50%) of the research.
- **[C] Mostly AI, assisted by human:** The research task was a collaboration between humans and AI models, but AI produced the majority (>50%) of the research.
- **[D] AI-generated:** AI performed over 95% of the research. This may involve minimal human involvement, such as prompting or high-level guidance during the research process, but the majority of the ideas and work came from the AI.

For each research stage where AI was involved, please also indicate the approximate level of iteration effort using the provided iteration macros. Count substantive attempts, prompts, runs, agent trajectories, or course corrections; do not count minor wording edits.

- **[Low]** : approximately 1-10 substantive attempts.
- **[Medium]** : approximately 10-100 substantive attempts, with some failed attempts or course corrections.
- **[High]** : approximately 100+ substantive attempts, with substantial exploration, failures, or trial-and-error.
- **[NA]** : not applicable because AI involvement was **[A]** .
- **[Unclear]** : the authors cannot reasonably estimate.

1. Hypothesis development.

Answer: **[C]**

Iteration Effort: **[Medium]**

Explanation: The human author selected the venue and the verifiable track and steered the topic toward the connected variant. A large language model then conducted the background literature search that established the absence of any published per- n connected-optima table, framed the gap, and proposed the verifier-proposer methodology. Idea generation and the supporting research were thus majority-AI, with human direction setting scope and goals.

2. Experimental design and implementation.

Answer: **[C]**

Iteration Effort: **[High]**

Explanation: An agentic coding assistant wrote the verifier, the CP-SAT flow model, the reproduced ASP encoding, and the experiment harness, and executed all runs. The human made the consequential strategic decisions (choice of connectivity encoding, sweep ordering, stopping criteria, and the decision to cross-check). Iteration was high: encoding bugs, a solver-API misreading, core oversubscription crashes, and a failed simulated-annealing design were diagnosed and corrected over many runs.

3. Analysis of data and interpretation of results.

Answer: **[C]**

Iteration Effort: **[Medium]**

Explanation: The AI organized run logs, derived the ceiling-matching argument, computed the connectivity penalties and bounds, and produced the solver comparison and the diagnosis of the simulated-annealing failure. The human reviewed and validated interpretations, particularly the soundness of the optimality claims and the honesty of the labeling.

4. Writing.

Answer: **[C]**

Iteration Effort: **[Medium]**

Explanation: The manuscript text, tables, and narrative were drafted by a large language model from the experiment log and were revised under human direction and review. The human set the framing priorities, checked all claims against the verified results, and is responsible for the final content.

301 **AI System Documentation**

302 **5. AI systems used.**

303 Description: A single frontier large language model was used in three roles: (i) conversational research
304 and ideation; (ii) an agentic coding assistant with file-system and code-execution access that engineered
305 the verifier, the exact engines, and the experiment harness and ran all experiments; and (iii) manuscript
306 drafting. The exact solving used a standard off-the-shelf CP-SAT constraint solver and a standard
307 answer-set-programming solver, both invoked by the agent; these are conventional combinatorial tools,
308 not AI systems. Specific product and model names are omitted here per the submission guidelines and
309 will be provided in the camera-ready version.

310 **6. Observed AI Limitations.**

311 Description: We observed several concrete failure modes. The model reproduced a constraint encoding
312 with a silent semantic error (an aggregate that failed to count neighbors), which produced no valid
313 solutions and was detected only by the independent verifier. It initially misread a solver’s optimality
314 signal, briefly mislabeling a proven result as unproven. It oversubscribed CPU cores in a way that
315 crashed the solver on the target hardware. And its first simulated-annealing design made no valid
316 moves on a tightly constrained problem, requiring a redesign that time did not permit. In each case
317 the deterministic verifier or an internal consistency check, rather than the model’s own confidence,
318 surfaced the error—reinforcing the value of grounding generative components in an exact checker.

Conference For AI Scientists 2026 - Reproducibility and Responsibility Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state proven optimality only for $n \in \{8, 9, 10, 11\}$ and label all other values best-found with bounds, matching Table 1.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 discusses the proof horizon at $n = 11$, the unclosed gaps for $n \geq 13$, the simulated-annealing failure, the single-effort implementation risk, and the dependence on published unconstrained optima.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions.
- The authors should reflect on the scope of the claims made and the factors that influence performance.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale.
- Reviewers will be instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Proposition 1 is stated with its assumptions (correctness of the published unconstrained optima and of the verifier) and proved; the remaining optimality results are computational (exhaustive search), cross-checked between two independent solvers.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All theorems, formulas, and proofs should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- Proofs can appear in the main paper or supplemental material with a proof sketch for intuition.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

Answer: [Yes]

Justification: We give the exact encodings (flow connectivity and the corrected ASP program), solver versions, random seeds, thread counts, and per- n time budgets, and release the verifier and all witness JSONs in the submission format.

Guidelines:

- The answer NA means that the paper does not include experiments.
- A No answer will not be perceived well: making the paper reproducible is important.
- Authors should describe steps taken to make results reproducible or verifiable.
- Authors are welcome to describe the particular way they provide for reproducibility.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results?

Answer: [Yes]

Justification: The verifier, the solver encodings, and every witness pattern are released (anonymized) as supplementary material with run instructions; the witnesses are in the official machine-checkable format.

Guidelines:

- The answer NA means that the paper does not include experiments requiring code.
- “No” is acceptable, but releasing code and data is encouraged.
- At submission time, anonymized versions should be released to preserve anonymity.
- Instructions should contain the exact command and environment needed to reproduce the results.

6. Experimental setting/details

Question: Does the paper specify all the training and test details necessary to understand the results?

Answer: [Yes]

Justification: Sections 3–4 and Table 1 specify the encodings, the objective, solver seeds and thread counts, and the time budgets per box.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented to a level of detail necessary to appreciate the results.
- Full details can be provided with the code, in an appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [NA]

Justification: The primary results are exact, deterministic, verifier-checked combinatorial facts (proven optima and validated witnesses), not statistical estimates, so error bars do not apply; for the stochastic proposer we report the best verified result over documented seeds.

Guidelines:

- The answer NA means that the paper does not include experiments.
- Authors should answer “Yes” if results are accompanied by error bars, confidence intervals, or significance tests.
- The factors of variability captured by the error bars should be clearly stated.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources needed to reproduce the experiments?

Answer: [Yes]

Justification: All experiments ran on a single fanless laptop using four performance cores and 24 GB of unified memory; per- n wall-clock times are reported, and total compute is modest (minutes per proven box).

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers, memory, and storage.
- The paper should provide per-run and total compute estimates.

9. Research Ethics

424 Question: Does the research conducted in the paper conform with the NeurIPS Code of Ethics, except
 425 where CAISc's AI authorship policies apply?

426 Answer: [\[Yes\]](#)

427 Justification: The work is purely combinatorial, involves no human subjects or personal data, and
 428 follows the Code of Ethics; the only deviation is the conference's own AI-authorship policy, which we
 429 follow as specified.

430 Guidelines:

- 431 • All submissions should adhere to the NeurIPS Code of Ethics, except where CAISc's AI
- 432 authorship policies apply.
- 433 • The answer NA means the authors have not reviewed the Code of Ethics.
- 434 • If the authors answer No, they should explain the special circumstances.

435 **10. Broader impacts**

436 Question: Does the paper discuss both potential positive societal impacts and negative societal impacts
 437 of the work performed?

438 Answer: [\[Yes\]](#)

439 Justification: Positive impacts include a reproducible, verifier-grounded template for AI-assisted
 440 combinatorial discovery; potential negative impacts are limited to the general dual-use nature of
 441 stronger automated optimization and agentic tooling, mitigated here by the emphasis on machine-
 442 checkable artifacts.

443 Guidelines:

- 444 • The answer NA means that there is no societal impact of the work performed.
- 445 • If the authors answer NA or No, they should explain why.
- 446 • Examples of negative impacts include malicious or unintended uses, fairness, privacy, and
- 447 security considerations.
- 448 • If there are negative impacts, authors could discuss mitigation strategies.